

Document number: P0116R0
Date: 2015-09-25
Project: Programming Language C++, Library Evolution Working Group
Reply-to: Robert Kawulak <Robert.Kawulak@gmail.com>

Boolean conversion for Standard Library types

I. Table of Contents

- II. [Introduction](#)
- III. [Motivation](#)
- IV. [Impact on the Standard](#)
- V. [Design Decisions](#)
- VI. [Technical Specifications](#)

II. Introduction

The paper proposes addition of explicit `bool` conversion operator to the following Standard Library types:

- `std::bitset`
- `std::chrono::duration`
- `std::complex`

It also discusses possibility of adding the operator to other types.

III. Motivation

Types providing a boolean conversion operator offer a convenient way to check whether an object is in some kind of type-specific invalid or “empty” state. With the addition of explicit conversion operators and contextual conversions to C++11, such operators also have much less chance to cause confusion. Some built-in and Standard Library types provide boolean conversion, but there are still some types in the Standard Library that lack this feature despite its potential usefulness.

For example, it's been a common situation for the author to need to check a `std::chrono::duration` value for being non-zero. Currently, this can be achieved like this:

```
std::chrono::seconds delay = get_delay();
if ( delay != std::chrono::seconds::zero() )
{
    // there is some delay, handle it...
}
```

Or, alternatively, in a less verbose way:

```
std::chrono::seconds delay = get_delay();
if ( delay.count() )
{
    // there is some delay, handle it...
}
```

The proposed addition would allow for expressing the same in a shorter and more readable way, for example:

```
std::chrono::seconds delay = get_delay();
if ( delay )
{
    // there is some delay, handle it...
}
```

Or even:

```
if ( std::chrono::seconds delay = get_delay() )
{
    // there is some delay, handle it...
```

}

IV. Impact on the Standard

The proposal describes an extension to some Standard Library types that should generally be safe and have no impact on existing code. However, it might be a breaking change in rare cases, like in code explicitly checking for existence of `bool` conversion operator in Standard Library types.

V. Design Decisions

The author searched the Standard Library for types that have a notion of invalid or “empty” state but lack a `bool` conversion operator. The types found are grouped into two categories: [proposed](#) and [possible](#).

Note that these lists lack string, container and range types since there already was a [proposal](#) to add a `bool` conversion operator to them and it has been [rejected](#).

Proposed

These types could have a `bool` conversion operator with natural semantics and their usability would clearly benefit. The author proposes to add the operator to them. The types and the corresponding operator's semantics are presented in the table below.

type	operator <code>bool ()</code> semantics
<code>std::bitset</code>	<code>count() != 0</code>
<code>std::chrono::duration</code>	<code>*this != zero()</code>
<code>std::complex</code>	<code>real() != T() imag() != T()</code>

Possible

These types have some kind of invalid or “empty” state and technically could have a `bool` conversion operator, but the author is not certain that the semantics are not potentially confusing or that there is obvious usability gain. Therefore the types are currently not included in the proposed types list, but the author is open to include those that the Committee would find worthwhile.

type	operator <code>bool ()</code> semantics
<code>std::weak_ptr</code>	<code>!expired()</code>
<code>std::sub_match</code>	<code>matched</code>
<code>std::match_results</code>	<code>ready()</code>
<code>std::thread</code>	<code>get_id() != id()</code>
<code>std::thread::id</code>	<code>*this != id()</code>
<code>std::future</code>	<code>valid()</code>
<code>std::shared_future</code>	<code>valid()</code>
<code>std::packaged_task</code>	<code>valid()</code>

VI. Technical Specifications

The proposed wording is relative to the [N4527](#) working draft. It only includes changes related to types listed in [Proposed](#) section.

- Change § 20.6 p. 1 [template.bitset] as follows:

```
...
bool any() const noexcept;
bool none() const noexcept;
explicit operator bool() const noexcept;
...
```

- Change § 20.6.2 [bitset.members] as follows:

```
...
bool any() const noexcept;
45 Returns: count() != 0
```

```

bool none() const noexcept;
46 Returns: count() == 0
explicit operator bool() const noexcept;
47 Returns: count() != 0
...

```

- Change § 20.12.5 p. 1 [time.duration] as follows:

```

...
// 20.12.5.2, observers:
constexpr rep count() const;
constexpr explicit operator bool() const;
...

```

- Change § 20.12.5.2 [time.duration.observer] as follows:

```

20.12.5.2 duration observers [time.duration.observers]
constexpr rep count() const;
1 Returns: rep_.
constexpr explicit operator bool() const;
2 Returns: *this != zero().

```

- Change § 26.4.2 [complex] as follows:

```

...
constexpr T imag() const;
void imag(T);
constexpr explicit operator bool() const;
...

```

- Change § 26.4.3 [complex.special] as follows:

```

...
constexpr float imag() const;
void imag(float);
constexpr explicit operator bool() const;
...
constexpr double imag() const;
void imag(double);
constexpr explicit operator bool() const;
...
constexpr long double imag() const;
void imag(long double);
constexpr explicit operator bool() const;
...

```

- Change § 26.4.4 [complex.members] as follows:

```

...
constexpr T imag() const;
Returns: The value of the imaginary component.
void imag(T val);
Effects: Assigns val to the imaginary component.
constexpr explicit operator bool() const;
Returns: real() != T() || imag() != T().

```